Ajuste de parámetros

La retropropagación es el método que se utiliza para calcular un gradiente para el caso de las redes neuronales. ¿Por qué un método complicado en vez de tratar de obtener la derivada analíticamente? Porque, generalmente, la cantidad de parámetros que posee una red neuronal es abrumadora: miles e incluso millones de parámetros tienen que ser ajustados, y obtener un gradiente de manera convencional es inviable computacionalmente.

Preámbulo

Ya se estableció que el gradiente es un vector el cual apunta en la dirección de mayor crecimiento y su magnitud es la tasa de crecimiento de una función. Y si bien la idea geométrica es fascinante, visualizar un vector apuntando a una dirección en un espacio de cientos o miles de dimensiones no es intuitivo. En vez de eso, veamos la siguiente perspectiva:

Se tiene el siguiente gradiente evaluado en un punto (x_0, y_0) para una función $z = f(x, y)$:

$$\nabla f(x_0, y_0) = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

Este gradiente, expresado en sus componente rectangulares, tiene las propiedades descritas del gradiente. Pero, si se nos pregunta qué dirección se debería tomar tal que este tenga una mayor influencia en el resultado de la función, el gradiente nos indica que cualquier movimiento en dirección del eje x será 3 veces mayor que cualquier movimiento en el eje y.
Cambios realizados en el valor de x tienen mayor influencia que en y, así que si lo que se quiere es ajustar los valores de la función para

minimizarla; suena más razonable realizar esos ajustes en x, ya que esta variable es más significativa.

Esta perspectiva aplica para un vector de dos dimensiones como para uno de cientos o miles de dimensiones. Si el valor del gradiente en un componente es muy grande, entonces este tiene mayor impacto en el resultado de la función; y si un valor es muy pequeño, entonces su contribución al cambio de la función es menor.

La idea de retropropagación

El gradiente no es más que una forma de ver qué tanta influencia tienen los parámetros en el resultado de la función. Hay parámetros con mayor influencia que otros, y la idea es ajustar estos parámetros en función de su influencia sobre el resultado de la función de costo, para así minimizar la función de costo. Es por eso que el gradiente es sumamente importante para el aprendizaje de la red.

Ya establecimos que obtener el gradiente de forma normal no es posible, al menos no eficientemente. Entonces, para obtener el gradiente negativo (para saber la dirección de mayor decrecimiento) se realiza lo siguiente:

1. Se toma una entrada puntual y se obtiene una salida en la última capa.
2. Se realiza una comparación del resultado esperado con el del resultado obtenido por la red neuronal. Esta comparación es el *error* determinado por la *función de costo*.
3. Se observa como cada paso del cálculo de la red neuronal contribuye al error. Este error se propaga desde la capa de salida hacia las capas anteriores, básicamente midiendo cuánto contribuyó cada neurona al error final.
4. Entre mayor sea la contribución al error, se opta por un ajuste grande de los pesos y sesgos que le suceden a la neurona; y a

menor influencia en el error, el ajuste es más pequeño.

5. Se lleva el seguimiento de los ajustes a realizar por cada entrada puntual, y el ajuste total a realizar es el promedio de los ajustes puntuales. Este de aquí es la estimación del gradiente de la función de costo.
6. Luego, se usa el negativo del gradiente para usar un ajuste de los pesos y sesgos por medio del descenso de gradiente.

Regla de la cadena

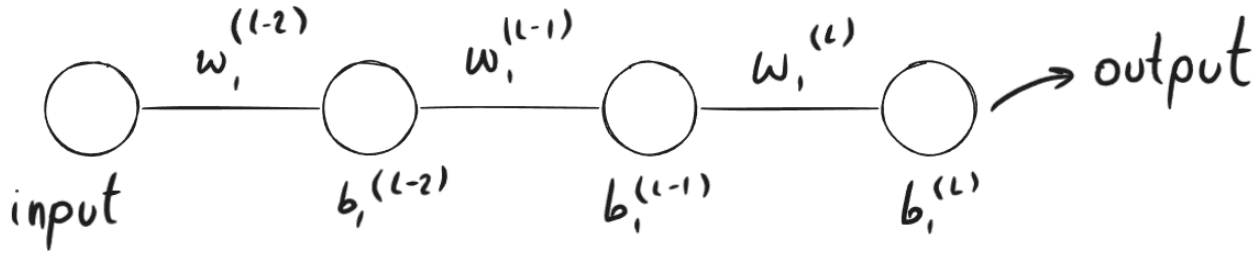
En la introducción, se mencionó cómo la estructura en capas de la red neuronal era útil pues permite subdividir problemas en operaciones intermedias, de manera similar a una composición de funciones. Esta idea es más fuerte, pues en esencia, la **red neuronal es una composición de funciones**.

> Composición de funciones
>
> Los valores de activación de una capa contribuye a las de la siguiente; u otra forma de verlo es que, la capa de salida de salida se encuentra en función de los valores de activación de la capa anterior; y esta penúltima capa está en función de los valores de activación de las neuronas de la antepenúltima capa... Y así hasta llegar a la entrada inicial.

Ejemplo introductorio

Para comprender la matemática detrás de la retropropagación, partiremos de un ejemplo muy sencillo: Una red neuronal de cuatro capas, una neurona en cada capa. Esta red neuronal, por lo tanto, posee tres pesos y tres sesgos; pues la primera neurona simplemente es la neurona de entrada. La notación de las capas empieza el conteo desde la capa de salida como l, por lo que cada elemento, como los

sesgos y los pesos tienen un superíndice $(l-k)$ para marcar la capa a la que corresponden.



Queremos encontrar el gradiente de la función de costo C para esta red neuronal, la cual tendría una estructura similar a esta:

$$\nabla C = \begin{bmatrix} \frac{\partial C}{\partial w_1^{(l)}} \\ \frac{\partial C}{\partial b_1^{(l)}} \\ \frac{\partial C}{\partial w_1^{(l-1)}} \\ \frac{\partial C}{\partial b_1^{(l-1)}} \\ \frac{\partial C}{\partial w_1^{(l-2)}} \\ \frac{\partial C}{\partial b_1^{(l-2)}} \end{bmatrix}$$

Recordando, la **función de costo** planteada es el error cuadrático medio (MSE), el cual es un promedio del error cuadrático de todos los datos puntuales. Pero, de momento, nos centramos en un dato puntual para obtener un costo C_0.

$$C_0 = (y - a^{(l)})^2$$

En esta notación, $a^{(l)}$ es el valor de activación de la última capa, y por tanto $a^{(l-k)}$ representa los valores de activación de la capa k-ésima después de la de salida. Nuevamente, hay que recordar que los valores de activación son el resultado de una función de activación $f(\cdot)$ a la cual ha sido evaluado el producto matricial del vector valores de

activación de la neurona anterior $a^{(l-1)}$, la matriz de pesos $w^{(l)}$; y luego sumar el resultado con el vector de sesgos $b^{(l)}$

$$a^{(l)} = f(w^{(l)}a^{(l-1)} + b^{(l)})$$

Para tener una notación más simple (y para seguir la idea de la bibliografía), vamos a comprimir la expresión que se evalúa en la función de activación como $z^{(l)}$.

$$\begin{aligned} a^{(l)} =& f(z^{(l)}) \\ z^{(l)} =& w^{(l)}a^{(l-1)} + b^{(l)} \end{aligned}$$

Vemos entonces como hay una relación de dependencia entre los sesgos, los pesos, y los valores de activación de la capa anterior con el costo de este dato puntual; una relación que consta de una composición de funciones:

$$C_0 \circ a^{(l)} \circ z^{(l)}$$

Pasemos a ver la influencia que tiene $w_1^{(l)}$ (el único peso en la capa) en el costo puntual C_0. Para hacerlo, usaremos derivadas parciales y la regla de la cadena. Si bien para este caso en particular, usar derivadas parciales no es diferente de usar derivación normal, tiene sentido para redes con más capas. La derivada de C_0 con respecto a $w_1^{(l)}$ es la siguiente:

$$\frac{\partial C_0}{\partial w_1^{(l)}} = \frac{\partial C_0}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial w_1^{(l)}}$$

Estas son derivadas que se pueden sacar analíticamente para tener una fórmula explícita:

$$\frac{\partial C_0}{\partial w_1^{(l)}} = 2(y - a^{(l)})f'(z^{(l)})a^{(l-1)}$$

Recordar que $f(\cdot)$ es una función de activación, que puede ser cualquier función no lineal. Si la función de activación fuera la

sigmoide, la derivada es sencilla:

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

Y, en el caso de ReLU, la cual es una función que no es diferenciable en 0, su derivada al final es la función escalón. Que la derivada de ReLU se evalúe en exactamente cero es muy improbable, y al final, no es de mucho impacto tomar una decisión arbitraria en este valor originalmente indefinido.

Nótese que esto es solo para obtener la tasa de cambio entre el **costo puntual** C_0 y el peso $w_1^{(l)}$, no es la tasa de cambio entre el **costo total**. Sin embargo el promedio de la tasa de cambio para todos los datos puntuales es igual a la derivada de la función de costo total C respecto a $w_1^{(l)}$. Este proceso es algo que hacemos con todos los componentes del gradiente.

$$\frac{\partial C}{\partial w_1^{(l)}} = \frac{1}{n} \sum_{i=0}^{n} \frac{\partial C_i}{\partial w_1^{(l)}}$$

Ahora, queremos ver cómo influye el sesgo $b_1^{(l)}$ en el costo puntual C_0. La derivada resultante es muy similar a la del peso $w_1^{(l)}$, la cual es:

$$\frac{\partial C_0}{\partial b_1^{(l)}} = \frac{\partial C_0}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial b_1^{(l)}}$$

Esto hace que la fórmula sea muy parecida a la anterior. De hecho, es más sencilla que la anterior ya que la derivada parcial de $z^{(l)}$ respecto a $b_1^{(l)}$ es 1.

$$\frac{\partial C_0}{\partial b_1^{(l)}} = 2(y - a^{(l)}) f'(z^{(l)})$$

Ahora viene lo curioso. La tasa de cambio con respecto al valor de activación de la capa anterior $a^{(l-1)}$.

$$\frac{\partial C_0}{\partial a^{(l-1)}} = \frac{\partial C_0}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial a^{(l-1)}} = 2(y - a^{(l)}) f'(z^{(l)}) w^{(l)}$$

La derivada del costo puntual C_0 con respecto al valor de activación $a^{(l-1)}$ son los pesos $w^{(l)}$. Pero, hay que recordar que $a^{(l-1)}$ no es una variable independiente, está en función de $z^{(l-1)}$, que a su vez está en función de los pesos $w^{(l-1)}$, los sesgos $b^{(l-1)}$, y los valores de activación $a^{(l-2)}$. Esta recursión ocurre hasta llegar a la capa de entrada. Entonces, la regla de la cadena aumenta conforme nos profundizamos más en las capas; pero las derivadas son similares a las ya encontradas, solo utilizando los valores de las capas anteriores. Entonces, para las derivadas del peso $w_1^{(l-1)}$ y del sesgo $b_1^{(l-1)}$, se tendría lo siguiente por regla de la cadena:

$$\frac{\partial C_0}{\partial w^{(l-1)}} = \frac{\partial C_0}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial a^{(l-1)}} \frac{\partial a^{(l-1)}}{\partial z^{(l-1)}} \frac{\partial z^{(l-1)}}{\partial w_1^{(l-1)}} = 2(y - a^{(l)}) f'(z^{(l)}) w^{(l)} f'(z^{(l-1)}) a^{(l-2}$$

$$\frac{\partial C_0}{\partial b^{(l-1)}} = \frac{\partial C_0}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial a^{(l-1)}} \frac{\partial a^{(l-1)}}{\partial z^{(l-1)}} \frac{\partial z^{(l-1)}}{\partial b_1^{(l-1)}} = 2(y - a^{(l)}) f'(z^{(l)}) w^{(l)} f'(z^{(l-1)})$$

Notación delta

Observar que en este caso particular, el término $2(y - a^{(l)}) f'(z^{(l)}) w^{(l)}$ se repite en cada derivada. De hecho, si queremos calcular las derivadas de la capa $(l-2)$ veremos que el siguiente término se repite:

$$2(y - a^{(l)}) f'(z^{(l)}) w^{(l)} f'(z^{(l-1)}) w^{(l-1)}$$

Observaremos que el término que se repite son las derivadas con respecto a la activación de la capa (l). En el caso de las derivadas de los parámetros de la capa $(l-1)$ siempre se realiza un producto con el término correspondiente la influencia de los valores de activación de la capa (l) con respecto al costo:

$$\frac{\partial C_0}{\partial a^{(l)}} = 2(y - a^{(l)}) f'(z^{(l)}) w^{(l)}$$

Y, en el caso de las derivadas con respecto a la capa $(l-2)$, el término repetido corresponde al cambio del costo con respecto a la activación de la capa $(l-1)$:

$$\frac{\partial C_0}{\partial a^{(l-1)}} = 2(y - a^{(l)}) f'(z^{(l)}) w^{(l)} f'(z^{(l-1)}) w^{(l-1)}$$

Esto muestra un comportamiento recursivo y, al computar, lo mejor sería no tener que hacer la misma operación una y otra vez para ahorrar tiempo y memoria. Entonces, una forma en la que este proceso se ve más evidente es por medio de la **notación delta**. Esta notación hace lo siguiente:

$$\delta^{(l)} = \frac{\partial C_0}{\partial z^{(l)}} = \frac{\partial C_0}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}}$$

La razón por la cual $\delta^{(l)}$ es para referirse al cambio del costo C_0 con respecto al proceso matricial $z^{(l)}$ como al error de la capa (l). Entonces, usando esta definición, veremos que la influencia del costo C_0 con respecto a la activación de la capa $(l-1)$ es:

$$\frac{\partial C_0}{\partial a^{(l-1)}} = \delta^{(l)} w^{(l)}$$

Sin embargo, ya que no tenemos influencia directa sobre $a^{(l-1)}$, o sobre $a^{(l-k)}$ en todo caso, sino que siempre estamos en búsqueda de la influencia del costo con respecto a los pesos $w^{(l-k)}$ y sesgos $b^{(l-k)}$... ¿Tiene sentido usar derivadas con respecto a $a^{(l-k)}$? La notación delta acorta esto, haciendo lo siguiente:

$$\delta^{(l-1)} = (\delta^{(l)} w^{(l)}) \cdot f'(z^{(l-1)})$$

Caso general (multivariable)

El ejemplo introductorio utilizaba una neurona por capa, y si bien se ha querido siempre tratar siempre a cada capa con un vector unidimensional en la explicación, para no alejarse tanto del uso

práctico... Hay ciertos detalles que se tienen que mencionar. Entre ellos, la notación de los sesgos, pesos, y valores de activación:

- Cuando se habla de un sesgo $b_j^{(l)}$, nos referimos al sesgo con índice j que se encuentra en la capa (l).
- Cuando se habla de un peso $w_{jk}^{(l)}$, nos referimos al peso que de la conexión entre la neurona k-ésima de la capa $(l-1)$ con la neurona j-ésima de la capa (l).
- Cuando se habla del valor de activación $a_j^{(l)}$, nos referimos al valor de activación de la neurona j-ésima de la capa (l).

En el caso anterior, se llegó a la siguiente fórmula para la notación delta:

$$\delta^{(l-1)} = (\delta^{(l)} w^{(l)}) \cdot f'(z^{(l-1)})$$

Esta ecuación es correcta para el caso especial del ejemplo introductorio, es decir, un caso escalar. El caso multivariable requiere un par de ajustes para funcionar, pero el resultado no cambia mucho en estructura:

$$\delta^{(l-1)} = (\mathbf{w}^{(l)})^\intercal \, \delta^{(l)} \odot f'(\mathbf{z}^{(l-1)})$$

Los cambios hechos son:

- El error $\delta^{(l)}$ se multiplica con la transpuesta de $\mathbf{w}$. Una forma sencilla de responder porqué, es que el proceso de retropropagación va en dirección opuesta al procesamiento de la red neuronal.
- Se realiza un producto especial entre el resultado de $(\mathbf{w}^{(l)})^\intercal \, \delta^{(l)}$ y $f'(\mathbf{z}^{(l-1)})$. Este producto es conocido como producto componente a componente, o también llamado **producto de Hadamard**.
  Este producto simplemente multiplica los componentes correspondientes de los vectores, asumiendo que estos tengan el

mismo tamaño. Naturalmente, el vector resultante tendrá la misma dimensión.